

LeNet-5 FPGA Project: Complete Technical Explanation

This document explains, in plain language, everything done during the HLS (High-Level Synthesis) phase of this project — what each technique means, why it was needed, what happened when we tried it, and what the results mean. It's written so that someone with no FPGA background can follow the whole story from start to finish.

Part 1: The Absolute Basics

What is HLS, and why does this project need it?

Your Python/Keras model runs on a CPU, which executes instructions one at a time (or a handful in parallel, at most). An FPGA is different: it's a piece of hardware you can configure to have thousands of tiny calculators all working *at the same time*. The challenge is that FPGAs are normally programmed in languages like Verilog or VHDL, which describe *circuits*, not *programs* — a much harder skill to learn from scratch.

High-Level Synthesis (HLS) is a compiler that takes C/C++ code and automatically converts it into that circuit-level hardware description. You write something that looks like a normal C++ function; the HLS tool (Vitis HLS, in our case) turns it into real digital logic. This is why every one of our building blocks — `conv_c1`, `pool_s2`, `dense_c5`, etc. — is written as plain-looking C++ functions, even though the end goal is a physical circuit.

What is a "pragma"?

A pragma is a special comment-like instruction, written as `#pragma HLS ...`, that doesn't change what the C++ code *computes* — it changes *how the hardware is built* to compute it. Think of it as a set of construction instructions layered on top of a blueprint: the blueprint (your C++ logic) stays the same, but the pragma tells the builder "put these two walls next to each other" or "build 4 of these rooms instead of 1."

What is "Initiation Interval" (II)?

This is the single most important number in this whole project, so it's worth being precise about it.

Imagine a factory assembly line. Each station does one step of building a car. If a new car can enter the line every 1 minute, the line's "interval" is 1 minute — regardless of how long it takes any single car to travel all the way through.

II is exactly this, but for hardware loops. If a loop has `II = 1`, it means: a new iteration of the loop *starts* every single clock cycle, even though any *one* iteration might take several cycles to fully finish (just like a car takes many minutes to travel the whole assembly line, even though a new car enters every 1 minute).

- **Lower II is better** — it means the hardware can start new work more often, so it finishes processing a whole image faster.
- **II = 1 is the theoretical best case** for a simple loop — a new result started every single cycle.
- Our whole optimization story, for almost every layer, has been about **discovering why II was stuck at some higher number, and whether/how we could bring it down.**

What is "Latency"?

The total number of clock cycles a function takes to run from start to finish, once. Latency and II are related but different: II tells you how *often* new work starts; Latency tells you how long the *whole job* takes

overall (which depends on II, plus how many loop iterations there are, plus some fixed startup/drain overhead).

What does "DSP", "FF", "LUT", "BRAM" mean?

These are the four basic resources every FPGA is built from — think of them as the raw construction materials, and every design we build "spends" some amount of each:

- **DSP (Digital Signal Processor)**: a small, dedicated hardware block built specifically for fast multiplication and addition. Every multiply-accumulate (MAC) operation in a neural network wants to use one of these if possible, since they're much faster and more efficient than building a multiplier out of general-purpose logic.
- **LUT (Look-Up Table)**: the basic general-purpose building block of an FPGA. Almost any small logical function (comparisons, simple math, control logic) gets built from LUTs.
- **FF (Flip-Flop)**: a single bit of memory — a register. Anything that needs to "remember" a value from one clock cycle to the next uses flip-flops. More parallelism and more pipeline stages generally means more flip-flops are needed to hold intermediate values.
- **BRAM (Block RAM)**: small, dedicated on-chip memory blocks, used to store larger arrays (like a full image) more efficiently than using individual flip-flops for every value.

Every FPGA chip (ours is a Xilinx Zynq-7020) has a *fixed, limited* supply of each of these. Part of good hardware design is using as few of them as possible while still hitting your performance target — using every single DSP/LUT/FF available isn't a badge of honor, it just means there's no room left for anything else (like the rest of the network).

What does "Fmax" mean, and why 100MHz?

Fmax is the fastest clock speed the synthesized hardware can reliably run at, as estimated by the tool. We picked a **target of 100MHz (a 10-nanosecond clock period)** at the very start of this project as a conservative, standard choice for this FPGA chip, and kept it fixed

throughout every experiment — this is important, because it means every comparison in this document (this design vs. that design) is a fair one: same target speed every time, only the design itself changes.

Part 2: The Pragmas We Actually Used, Explained One at a Time

#pragma HLS PIPELINE II=1

What it does: tells the HLS tool "make this loop start a new iteration every single clock cycle, no exceptions." This is a *demand*, not just a hint — the tool will restructure the hardware however necessary to try to satisfy it.

Important side-effect we discovered the hard way: `PIPELINE II=1` forces the tool to **completely unroll** (build separate, parallel hardware for) everything nested inside that loop. This bit us directly in the first attempt at C1's systolic design — putting `PIPELINE II=1` around what was supposed to be a slow, resource-saving loop instead forced it to build 150 parallel multipliers, defeating the entire point of that experiment. The fix was moving the pipeline pragma to a different, smaller loop, so only the intended part got the "must be II=1" treatment.

#pragma HLS ARRAY_PARTITION

What it does: normally, an array (like your weights or an image row) is stored in one memory block with only 1-2 "ports" — meaning only 1-2 values can be read out of it in a single clock cycle. If your calculation needs to read, say, 25 values from that array *at the same time* (like all 25 weights in a 5×5 kernel), a single memory block can't do that — it becomes a bottleneck. `ARRAY_PARTITION` splits one array into several independent smaller memories (or even individual registers), each with its own port, so many values can be read simultaneously.

complete dim=N: split every single element along dimension N into its own independent register. Best for small arrays (like our 150-element convolution weights) where full parallelism is cheap.

cyclic factor=N: split the array into N interleaved groups, rather than one register per element. Used for the pooling layers, where full "complete" partitioning would have been overkill for what was needed.

The critical lesson we learned (the hard way, more than once): you must identify *which* array is actually the bottleneck before partitioning it. In C1's very first optimization attempt, we partitioned the `weights` array and got **zero improvement** — because it turned out `input` was the real bottleneck, not weights. The way to know for sure is reading the actual warning message HLS prints, which literally names the array causing the problem (e.g. *"Unable to schedule 'load' operation... on array 'weights' due to limited memory ports"*).

#pragma HLS UNROLL

What it does: takes a loop and builds separate, independent hardware for *every* iteration, instead of reusing the same hardware over and over across cycles. If a loop runs 8 times and you fully unroll it, you get 8 physical copies of whatever's inside, all able to work simultaneously — at the cost of needing 8× the hardware resources.

#pragma HLS INTERFACE m_axi and #pragma HLS INTERFACE s_axilite

What these do: these aren't about making the *computation* faster — they're about connecting your custom hardware block to the rest of the system (the ARM processor and memory). Think of your `conv_c1` or `dense_c5` block as a specialist worker in a factory — these pragmas are what actually wire that worker into the factory's conveyor belts and telephone lines so it can receive materials and instructions.

- **m_axi** ("AXI Master") gives your hardware block its own direct connection to main memory (DDR RAM), so it can read/write large arrays (images, weights) on its own, without needing the

processor to hand it every single value one at a time.

- **s_axilite** ("AXI Slave, Lite version") is a small, simple control channel that lets the ARM processor start your hardware block, check if it's finished, and read/write small configuration values.

A serious mistake we made and fixed: you cannot combine `m_axi` with full `ARRAY_PARTITION complete` on the *same* array. Partitioning says "split this into many independent on-chip registers"; `m_axi` says "this data actually lives off-chip in DDR and streams in over one shared connection." These are contradictory instructions for the same array. Our first attempt at this produced **150+ redundant hardware connections** and took nearly 20 minutes just to generate a report (should take under a minute). The fix: keep the DDR-facing arrays *unpartitioned*, copy their contents into small *local* on-chip arrays first, do all the fast/parallel work on those local copies, then copy the results back out. This is a standard, well-known HLS pattern once you know to look for it.

Part 3: The "Accumulator Latency" Problem — The Single Biggest Recurring Lesson

This came up independently in **three different layers** (C1's convolution, C3's convolution, and every dense layer) — worth understanding thoroughly, since it's the project's central hardware-design finding.

The problem, explained with an analogy

Imagine you're adding up a long list of numbers one at a time on a calculator that takes 4 seconds to actually display each result: $2 + 3 =$ (wait 4 seconds) $\dots + 5 =$ (wait 4 seconds) $\dots + 7 =$ (wait 4 seconds), and so on. Even though *entering* each new number only takes an instant, you're stuck waiting for the calculator's display before you can add the next number, because each step needs the *previous* step's answer.

This is exactly what happens with a line of code like:

```
acc = acc + (input * weight);
```

repeated many times in a loop. A hardware floating-point adder isn't instant — ours took about 4-5 clock cycles internally to actually produce its answer. Since each loop iteration *needs* the previous iteration's answer before it can proceed, the loop is forced to wait, and **it gets stuck at roughly the adder's own latency** (we consistently measured this landing at $ll=4$ or $ll=5$), no matter what else you try.

Why array partitioning doesn't fix this

This is a completely different kind of bottleneck than the "not enough memory ports" issue partitioning solves. Partitioning fixes "I can't *read* enough values per cycle." Accumulator latency is "I *can* read the values fine, but I can't finish *adding* them fast enough, because each addition depends on the one before it." No amount of extra memory ports changes how long a single addition takes to compute.

The fix: splitting one long chain into several shorter, independent chains

Instead of one accumulator doing `acc = acc + x1; acc = acc + x2; acc = acc + x3; ...` all in one dependent sequence, we use **several separate accumulators** (we called this "PE_COUNT", short for "Processing Element count"), each handling a subset of the terms independently:

```
partial_sum[0] = x1 + x5 + x9 + ...
partial_sum[1] = x2 + x6 + x10 + ...
partial_sum[2] = x3 + x7 + x11 + ...
partial_sum[3] = x4 + x8 + x12 + ...
```

Since these four chains don't depend on each other at all, they can all make progress at the same time, and only get added together

(`partial_sum[0] + partial_sum[1] + ...`) once, right at the very end. This is directly inspired by the paper's own "8 PE systolic array" concept, and is exactly why we built a `PE_COUNT`-based partial-sum design for C1, C3, `dense_c5`, and `dense_f6`.

The surprising discovery: more `PE_COUNT` doesn't automatically mean lower `II`

We proved, by actually testing it (not just assuming), that increasing `PE_COUNT` beyond the adder's own latency (4, in our case) **does not** get you all the way down to `II=1`. We tested `PE_COUNT=4`, 5, and 6 on different layers and every single one landed at `II=4` — never lower.

The real reason (confirmed by careful reading of the tool's own scheduling report): when you write `#pragma HLS UNROLL` across the PE dimension, all the partial-sum updates happen *within the same loop iteration*. Each individual `partial_sum[p]` value still only gets revisited once per iteration — a "gap" of exactly 1 iteration between one update and the next, regardless of how many separate partial sums exist. Since a single iteration still needs to fully account for the adder's 4-cycle latency, `II = 4` is the floor no matter how many parallel chains you add. Adding more `PE_COUNT` only reduces the *total number of iterations needed* (since each iteration now does more work at once) — it doesn't change the *per-iteration* limit.

What this means in practice: once you hit this floor, throwing more hardware resources at the problem stops helping the metric that matters (`II`), while still costing more DSP, FF, and LUT for no further benefit. We confirmed this directly: `PE_COUNT=5` used 71% more DSP than `PE_COUNT=4` for only 20% more speed improvement, with *identical* `II` — so we kept `PE_COUNT=4` as the better choice.

Part 4: Layer-by-Layer Story

C1 — First Convolution Layer ($28 \times 28 \times 1$ input $\rightarrow 28 \times 28 \times 6$ output)

This was the very first layer built, and the one where every technique in this document was discovered for the first time (the rest of the layers benefited from lessons learned here).

1. **Baseline** (no pragmas at all): $II = 13$, meaning a new pixel's worth of computation only started every 13 clock cycles. Very slow.
2. **First fix attempt — partition weights**: no improvement at all (II stayed at 13). This taught us the critical lesson: you must find the *actual* bottleneck array before partitioning, not guess.
3. **Line buffer + full partitioning**: instead of trying to have instant access to the *entire* 28×28 image at once (wasteful — most of it isn't needed at any given moment), we built a "line buffer" that only keeps the most recent 5 rows of the image on hand at once (since a 5×5 kernel only ever needs 5 rows), and made *that* small buffer fully partitioned. Combined with also partitioning the output array (since 6 output channels were being written simultaneously), this achieved the ideal $II = 1$ — the best possible result — using 150 parallel multiply/add units.
4. **Systolic PE array (matching the paper's "8 PE" design)**: rather than 150 parallel MAC units, we tried to match the paper's approach of just 8 processing elements sharing the work over multiple cycles. First attempt accidentally rebuilt full parallelism (the PIPELINE $II=1$ placement mistake described in Part 2). Second, corrected attempt genuinely used only 2 physical multiply/add units (thanks to further sharing the tool found automatically) — a **75× reduction in DSP usage** — at the cost of II rising to 4 (the accumulator-latency floor described in Part 3) and a roughly 24% drop in maximum clock speed. This is a completely real, honest engineering trade-off: much cheaper hardware, meaningfully slower per-pixel throughput.
5. **AXI interface conversion (for connecting to the rest of the system)**: first attempt combined `m_axi` with full array partitioning on the same arrays, producing the 150+ redundant connections mistake described in Part 2. Fixed by copying data into local on-chip buffers first, and computing there instead.

6. **Vivado integration:** successfully built and validated a complete system diagram containing the ARM processor, the necessary connection hardware (AXI Interconnect), and our custom C1 hardware block — confirmed with no errors by Vivado's own validation check.

S2 — First Pooling Layer ($28 \times 28 \times 6 \rightarrow 14 \times 14 \times 6$)

Pooling has no weights and no multiplication at all — it's just "look at a small window of values, keep the largest one" (max pooling) or "average them" (average pooling).

- **A real bug was caught and fixed here:** the layer was initially built using *average* pooling, matching an earlier (tanh-based) version of the model, before the whole project standardized on the ReLU+MaxPool variant. This was corrected to genuine max pooling.
- **Resource result:** max pooling uses **zero DSPs** — since it only needs comparisons, not multiplication or addition, it's inherently cheaper than average pooling (which needed a divide-by-4 operation).
- **Optimization:** same idea as C1 — the 2×2 window needs to read 4 values from the input array at once, so the input needed partitioning (specifically, `cyclic factor=2` on the row and column dimensions). This achieved `II = 1`.

S4 — Second Pooling Layer ($10 \times 10 \times 16 \rightarrow 5 \times 5 \times 16$)

Built using the exact same proven max-pooling + partitioning pattern as S2, just resized for different input dimensions (since it processes C3's output, not C1's). Reaching `II = 1` on the very first attempt — no trial-and-error needed, since the fix was already known. This is a nice, concrete demonstration of the paper's central idea: a proven hardware design, reused across multiple layers.

C3 — Second Convolution Layer ($14 \times 14 \times 6 \rightarrow 10 \times 10 \times 16$)

- **Baseline:** $II = 5$ — but this time, the cause was different from C1's baseline bottleneck. C1's problem was memory ports (not enough simultaneous reads); C3's was the accumulator-latency problem described in Part 3, since C3's convolution sums over 150 terms per output ($5 \times 5 \times 6$) in one long dependent chain.
- **Attempted fix #1 — partial-sum split with a "flattened" index (PE_COUNT=6):** worked, and worked very well — latency dropped by $7.66\times$. But it came with a hidden, unwanted cost: converting a single loop-counter number back into three separate coordinates (row, column, channel) required genuine integer division hardware (since the kernel size 5 isn't a "round" binary number like 4 or 8), which used up far more DSPs and flip-flops than the partial-sum idea alone should have needed.
- **Attempted fix #2 — remove the division by using natural nested loops instead:** this *did* remove the expensive division hardware entirely (confirmed: DSP dropped right back down to normal levels). But it had its own hidden cost: removing that "flattened index" math also broke a different optimization the tool had been doing for free (merging multiple loops into one continuous pipeline), and the design ended up **$6.4\times$ slower** overall than fix #1, with a lower maximum clock speed too.
- **Decision:** we kept fix #1 (the version with the division hardware) as the actual final design, specifically **because it was the only one of the two that still met our 100MHz timing target** — fix #2, despite using less hardware, was measured at only 93MHz, meaning it would not actually run correctly at our intended clock speed. This is an important, general lesson: a "resource-cheaper" design is not automatically a "better" design if it fails to meet the required speed.

Dense Layers — C5 (400→120), F6 (120→84), Output (84→10)

Dense (fully-connected) layers are conceptually simpler than convolution — no sliding window, just "multiply every input by its own weight, add them all up, repeat for every output neuron." But they hit the *exact same* accumulator-latency problem as C3, for the same underlying reason (one long dependent addition chain per output).

- **C5 and F6:** both built with the proven `PE_COUNT=4` partial-sum split from the start (no baseline-then-fix cycle needed, since the fix was already known from C3/C1). Both landed at the same `II = 4` floor, confirming this really is a general property of floating-point accumulation, not something specific to convolution.
- **Output layer — a genuinely different structure, on purpose:** this layer uses "softmax" activation, which is different from every other activation in this project (ReLU) in one important way — it needs to know the value of *every* output neuron before it can correctly compute *any single one* (it works by comparing each neuron's value against the total). This meant it could not use the same simple "compute one value, then immediately apply activation, then move to the next" pattern the other layers use. We explicitly split it into two clearly separate stages: first, calculate all 10 raw numbers into a small temporary list; second, run the actual softmax steps (find the largest value, calculate exponentials, sum them, divide) as their own separate pass over that completed list. This produced a correct, clean design — not one continuously pipelined loop, but four short, purpose-built stages in sequence, which is the structurally correct shape for this specific kind of computation.
- **A genuine software bug was found and fixed here too:** the tool's version of the C++ standard library didn't include a specific function (`expf`) we initially used for the exponential calculation; switched to the equivalent, available `exp` function instead.

Part 5: Summary Table (All Layers)

Layer	Function	Final II	DSP	Notes
C1 (systolic)	1st Convolution	4	2	75x DSP reduction vs. fully-parallel version; matches paper's 8-PE concept

S2	1st Max Pooling	1	0	No multiplication needed at all
C3 (partial-sum)	2nd Convolution	4	45 (mostly index-decode hardware)	Kept despite higher DSP, because the "cheaper" alternative failed timing
S4	2nd Max Pooling	1	0	Reused S2's proven fix directly, no rediscovery needed
Dense C5	1st Fully-Connected	4	7	Same accumulator-latency floor as convolution layers
Dense F6	2nd Fully-Connected	4	11	Same floor again — confirms it's independent of layer size
Dense Output	Final classification	(4-stage design, not single II)	14	Softmax's global-normalization requirement needed a different structure entirely

Part 6: What This Demonstrates, Overall

1. **We independently rediscovered a real, general hardware-design principle** (that floating-point accumulation chains have a latency floor no amount of parallelism alone can remove) across

three completely different layer types, through actual measurement rather than assumption — and then correctly explained *why* it happens at a mechanical level.

2. **We correctly diagnosed and distinguished two different kinds of hardware bottleneck** — memory port contention (fixed by array partitioning) versus computational dependency (fixed by splitting into independent parallel chains) — and learned that applying the wrong fix to the wrong problem produces no benefit at all (C1's first weights-partition attempt).
 3. **We learned that "using fewer resources" and "being a better design" are not the same thing** — a design must be judged by whether it meets its timing requirement first, and resource cost second (C3's rejected division-free alternative).
 4. **We demonstrated the paper's central "reusable IP" claim directly**, not just by reading about it: S4 reused S2's exact fix with zero rework, and F6 reused C5's exact PE_COUNT=4 pattern with zero rework — proving that a properly-designed hardware template really can transfer cleanly across layers of the same type.
-

Part 7: Combining All Seven Layers Into One Network (`lenet5_top`)

Once every individual layer (C1, S2, C3, S4, C5, F6, Output) had its own working, optimized hardware design, the final step was chaining all seven together into one complete accelerator that takes in a 28×28 digit image and produces a 10-value classification result — the actual, complete LeNet-5 network, not just its individual pieces.

How the pieces were joined

A new top-level function, `lenet5_top`, was written that simply calls each of the seven already-built layer functions **in sequence**, passing each layer's output as the next layer's input — `conv_c1` feeds `pool_s2`, which feeds `conv_c3`, and so on down to `dense_output`.

This reused every layer's already-proven, already-optimized C++ code with no changes to the layers themselves.

Verifying correctness with a real image

Rather than testing with artificial all-ones data (as the individual layers' testbenches did), `lenet5_top` was tested with an actual real handwritten digit from the MNIST dataset and the actual trained weights from the project's Keras model. The hardware design's output was compared against the equivalent NumPy/Python calculation (the same reference code used throughout the software side of this project) — the two matched to within 0.00000000000065 (a difference of about 65 trillionths), which is exactly the tiny, expected rounding difference between two different valid orders of floating-point arithmetic, not a bug. Both the hardware design and the Python reference correctly identified the test image as the digit it actually was. This is strong, direct evidence that the combined hardware design computes the *same network*, correctly, end to end.

Attempt 1 — "Parallel processing" between layers, and why it was reverted

Following a suggestion from the project guide to explore parallel processing in Vitis/Vivado, a feature called `#pragma HLS DATAFLOW` was tried. This pragma lets different layers overlap in time across successive images — for example, letting the convolution layer start working on image #2 while the pooling layer is still finishing image #1, like stations on a factory assembly line all working at once on different items, rather than one station finishing completely before the next one starts anything.

This was found to be a genuinely different kind of parallelism than anything used before in this project — the earlier `PE_COUNT` work parallelizes the *arithmetic inside* one layer; `DATAFLOW` parallelizes *whole layers* against each other, across time.

The result: the design became too large to fit on the actual FPGA chip. Specifically, it needed 158% of the chip's available on-chip

memory (BRAM) and 140% of its available general logic (LUT) — meaning, quite literally, more physical hardware than the chip contains. This is a hard limit, not a performance shortfall: a design like this cannot be placed onto the chip at all, regardless of clock speed.

Why this happened, once diagnosed: DATAFLOW's overlap trick requires every piece of data crossing from one layer to the next to be "double-buffered" — kept in two copies, so one copy can be read by the next layer while a fresh copy is being written by the current one, without the two interfering. This makes complete sense for the actual *image data* flowing through the network, since a new image genuinely does arrive while the old one is still being processed. But the network's **weights** (the trained parameters) never change from one image to the next — there was no need to double-buffer them at all, yet they were being copied in fresh, in duplicate, for every single image, purely because their copy-in step happened to live inside the same parallel-managed region as the image data.

The decision: revert to a simple sequential design for the primary working version. Re-reading the original paper confirmed that this specific across-layer, across-image overlap is not something the paper itself describes or requires — the paper's own parallelism claims are about the *inside* of a single convolution layer (which this project had already built, via the line-buffer and PE_COUNT work). Given that, and given that a design which cannot physically fit on the chip cannot be evaluated or used at all, the pragmatic and technically correct choice was to remove DATAFLOW and run the seven layers simply one after another, and treat the parallel-processing attempt as a documented exploration rather than the final design.

Attempt 2 — the sequential (non-parallel) version

Removing DATAFLOW and re-running confirmed the diagnosis was correct: on-chip memory usage dropped from 158% down to 82% (comfortably fitting), simply by removing the unnecessary weight duplication. Correctness was reconfirmed unchanged — removing this pragma only affects *scheduling* (when things happen), never *what is computed*, so the same exact numeric result and correct prediction were produced again.

One resource problem remained: general logic usage (LUT) was still over budget, at 131% of the chip's capacity. This was traced to several contributing sources added together, not one single cause: the C3 layer's known index-decoding overhead (discussed in Part 4), the C1 systolic design's own logic, the final softmax layer's multi-stage structure, and — a new finding specific to this combined design — the overhead of the AXI "adapter" hardware needed for each individual memory connection. Since the combined design originally gave every one of its 12 separate data arrays (across all seven layers) its own individual AXI connection, and each individual connection carries its own fixed hardware cost, this added up to a meaningfully large and somewhat wasteful total. The fix being pursued is to share one AXI connection across multiple arrays *within the same layer*, cutting down the number of individual connections from 12 to closer to 7 (one per layer) and reducing that repeated fixed cost accordingly.

A genuine self-correction worth noting

While investigating why the C3 layer's timing got worse inside the combined design compared to running on its own, the first explanation offered was that `DATAFLOW` was the cause. After `DATAFLOW` was removed, the exact same timing regression was still present — proving that first explanation wrong, and the record was corrected accordingly rather than left standing. The real cause turned out to be a more general fact: when C3's code is called *as a helper function from inside a larger design* rather than being the *top-level* function on its own, the tool no longer applies one particular optimization (merging several nested loops into one continuous pipeline) that it had applied when C3 stood alone. This is a separate, still-open finding, worth investigating further once the on-chip-memory-fit problem is resolved — but it affects only *speed*, not whether the design fits on the chip at all, so it was correctly set aside as lower priority until the capacity problem is solved.

Closing the LUT gap, part 1 — sharing memory connections between arrays

Before finding the real, largest cause of the LUT overage, a smaller and

more straightforward fix was tried first. The combined design originally gave **12 separate individual memory connections** to the outside world — one for every single data array across all seven layers (each layer's inputs, weights, biases, and outputs, all separately wired to memory). Each one of these connections carries its own fixed hardware cost simply to exist, regardless of how much data actually flows through it — the equivalent of running a separate physical cable to every device in a room instead of using a shared cable and letting devices take turns.

Grouping several arrays *belonging to the same layer* under one shared connection instead of separate ones (going from 12 connections down to 7 — one per layer) removed a real, measurable amount of this fixed overhead, with no effect whatsoever on the actual computation or timing (confirmed: correctness and speed were both unchanged, only the LUT count dropped). This was a genuine, free improvement — but it turned out to only recover a modest fraction of the total overage, since a much larger cost was hiding elsewhere.

Closing the LUT gap, part 2 — the real cause, and a genuinely elegant fix

Digging further, the single largest remaining contributor to the LUT overage was traced to one specific design choice inside the C3 convolution layer — the same "split one long calculation into several shorter, independent pieces" technique described in Part 3 of this document (the `PE_COUNT` accumulator-splitting fix). That technique works by assigning a large counting number (0 through 149, one for each of C3's 150 multiply-and-add steps per output) and needing to convert that single number back into three separate pieces of information: which row of the filter, which column, and which input channel it corresponds to. The original way this conversion was done used **division and remainder arithmetic** — and building actual division circuitry in hardware is one of the most expensive things you can ask an FPGA to do, in terms of general-purpose logic (LUT) cost.

The fix, once correctly identified, turned out to be genuinely elegant rather than a compromise. Since the counting number only ever takes on one of 150 specific, completely predictable values, there was no need to *calculate* the row/column/channel answer at all — every

possible answer could simply be **written down once, in advance**, in three small lookup tables. Looking up a precomputed answer is a cheap, simple operation (the same kind of operation as reading any ordinary array), completely different in hardware cost from performing actual division. This preserved the *exact same* loop structure that had let the tool optimize the design well in the first place (merging several loops into one efficient, continuously-flowing pipeline) — the earlier "just remove the division" attempt, by contrast, had accidentally broken that same structure and made the design 6.4 times slower as an unwanted side effect (documented in Part 4 as the "attempted fix #2" for this same underlying issue, then affecting the C3 layer on its own before this combined-network stage was reached).

The result was measured, not assumed, before being trusted: the new lookup-table version was tested first against a version of C3 using genuinely different, non-uniform weight values (rather than the simpler all-identical-value test used elsewhere in this project) — specifically because a bug that scrambles which row/column/channel goes with which number could still, by coincidence, produce the exact right *total* if every value being added together happened to be the same. This stricter check confirmed the new version's answer matched the old version's answer exactly, with zero difference, ruling out that specific class of subtle bug before trusting the result.

With this fix in place: DSP usage for this part of the design dropped by about three-quarters, memory/flip-flop usage dropped by over 90%, and general logic (LUT) usage dropped by 80% — **at exactly the same speed, same efficiency, and same maximum clock speed as before**. This is an unusually clean outcome in hardware design, where cost and speed usually trade off against each other; here, the earlier division-based approach turned out to simply have been solving the coordinate-conversion problem in a needlessly expensive way, not because expense was required for that speed.

Final result: the complete network fits, and works

With this improved version of C3 substituted into the full seven-layer network, the whole combined design was re-verified and re-measured from scratch:

- **Correctness:** unchanged and reconfirmed — the same real MNIST test image, run through all seven hardware layers in sequence using the actual trained weights, still produces a result matching the Python software reference to within 0.0000000000066 (a difference far smaller than floating-point rounding noise), and still correctly identifies the digit.
- **Chip capacity — every resource now fits:**

Resource	Used	Chip's Total Capacity	Fits?
General logic (LUT)	37,974	53,200	Yes — 71% used
Multiply-hardware (DSP)	24	220	Yes — 11% used
Memory bits (FF)	39,688	106,400	Yes — 37% used
On-chip memory (BRAM)	232	280	Yes — 82% used

- **Speed:** the design meets its 100MHz timing target, estimated capable of running at about 103.5MHz — with the complete seven-layer calculation for one input image taking roughly 496,749 clock cycles (about 4.8 thousandths of a second per image at this clock speed).

This is the completed deliverable: a full, seven-layer LeNet-5 neural network, implemented as real, synthesizable FPGA hardware, verified for correctness against real data and real trained weights, and confirmed to physically fit within the resource budget of the target chip (Xilinx Zynq-7020) at the project's chosen 100MHz clock target.

A note on what this design prioritizes, and what was intentionally set aside

Worth being explicit about the shape of the final result, since "it works and it fits" is not the same claim as "it is the fastest possible design."

The seven layers run one after another (no overlap between layers, since the earlier `DATAFLOW` parallel-processing attempt was reverted for exceeding the chip's memory capacity, as described above). Two further optimizations remain open, identified but intentionally not pursued, since they were not required to meet this project's actual goal of a complete, correct, working network:

1. **C1's convolution layer still uses the same division-based coordinate-conversion approach that C3 used to use** — the same lookup-table fix that helped C3 significantly would very likely help C1 too, since it currently is the single largest remaining individual contributor to LUT usage in the combined design. This was not required to fit the chip's budget, so it was left as a known, well-understood possible future improvement rather than pursued for its own sake.
2. **Per-layer speed within the combined network has not been individually re-tuned** — for instance, C3 was observed to run somewhat slower as a called sub-function inside the combined design than it did when synthesized entirely on its own, for reasons connected to how the tool optimizes function calls versus top-level code (documented above). This affects overall speed, not correctness or chip capacity, and was set aside as a lower-priority item once the higher-priority capacity problem was solved.

Both are legitimate, clearly-understood directions for further work, not open questions about whether the current design is correct or complete.

Part 8: From Design to Real Hardware — Generating the Actual Bitstream

Everything described up to this point — Vitis HLS's synthesis, the resource percentages, the timing estimates — are the *tool's own predictions* of what a design will need and how fast it will run, calculated before any real physical circuit has been laid out. The final, genuinely conclusive step is asking Vivado to actually perform **place-and-route**: taking the complete design and working out exactly which physical

LUTs, flip-flops, and wires on the real chip each part of the design will occupy, and exactly how long signals will actually take to travel between them. The end product of this process is a **bitstream** — a file that, if loaded onto a real board, configures the FPGA's physical hardware to literally become the designed circuit. This is the only step that produces genuinely final, ground-truth numbers rather than estimates.

A real hardware constraint encountered along the way

Partway through this process, the build appeared to simply stop making progress for an extended period — investigated properly rather than assumed to be a normal delay, by directly checking whether the relevant background process was still doing real work (its own measure of computation time was checked twice, several seconds apart, and had not moved at all in that window — a reliable sign of a genuine stall, not just a slow but active calculation).

The cause, once diagnosed, was a real resource limitation of the development machine itself, not a mistake in the design: the machine performing this build had a relatively modest 7.7GB of memory available, and had been instructed to run four separate synthesis jobs *simultaneously* to save time — a completely reasonable choice for a machine with more memory, but one that caused all four jobs to compete for the same limited memory pool at once, most likely leading to the system thrashing (spending most of its effort swapping data in and out of memory rather than doing useful computation) rather than a true logical hang.

The fix: the stalled attempt was stopped, and the exact same build was relaunched running only **one** job at a time instead of four. This trades total wall-clock time (each piece of work now happens one after another instead of overlapping) for a much smaller memory footprint at any given moment — and this time, the same synthesis step that had previously stalled for over three and a half hours with zero progress completed successfully in about 22 minutes.

This is worth recording honestly as a genuine finding of the project: **implementing a network of this complete size is not just a matter of correct code, but also requires adequate development-machine**

resources — a real, practical constraint of doing FPGA work on modest hardware, separate from any question of whether the design itself was correct.

The result: a complete, valid, working bitstream

With the corrected, single-job approach, the full flow — logic synthesis, followed by placement, routing, and final bitstream generation — completed cleanly, with **zero design-rule-check errors** at every stage. The final bitstream file (`lenet5_system_wrapper.bit`, about 3.9 megabytes) was confirmed to exist and be valid.

Final, real, place-and-routed timing (not an estimate — this is what the actual physical circuit, as laid out on the real chip, is capable of):

Timing Metric	Result	Meaning
Worst setup slack (WNS)	+3.659 ns	Positive means the design meets its clock speed target with margin to spare
Worst hold slack (WHS)	+0.008 ns	Positive means it passes — but only just, with almost no safety margin
Failing timing paths	0	Every single timing requirement in the whole design is satisfied

The worst setup slack being comfortably positive confirms the design genuinely can run at its intended clock speed. The hold slack passing by only eight-thousandths of a nanosecond is worth being upfront about: it passed, but with essentially no cushion — a legitimate, documented characteristic of this specific implementation, worth re-checking if the design is ever rebuilt with different tool settings or a different random starting layout, since a different attempt could plausibly land on the wrong side of zero even though the underlying design has not changed at all.

Final, real, place-and-routed resource usage (the actual physical chip area consumed, measured after real placement — not the earlier per-layer HLS estimates):

Resource	Used	Chip's Total Capacity	Percentage
General logic (LUT)	29,403	53,200	55.3%
Memory bits (FF)	41,096	106,400	38.6%
Multiply-hardware (DSP)	24	220	10.9%
On-chip memory (BRAM)	119.5	140	85.4%

One detail worth explaining, since it might look surprising at first: the real, final LUT usage (55.3%) came in noticeably *lower* than the earlier individual-IP HLS estimate (71%). This is expected, not an inconsistency to be concerned about — Vivado's whole-system placement step is able to find and eliminate redundant logic *across* the boundaries between different layers' hardware in a way that HLS, working on the seven layers somewhat independently, could not see or take advantage of. A real, final, whole-system number coming in more efficient than the sum of its independently-estimated parts is a normal and genuinely reassuring outcome.

What this represents, completed

This is the conclusive result of the project's entire hardware-implementation effort: a **real, physically valid, timing-verified FPGA configuration file**, generated through the industry-standard tool flow (Vitis HLS for the individual layer designs, Vivado for full system integration and physical implementation), for a complete seven-layer LeNet-5 neural network — proven correct against real image data and real trained weights earlier in this document, and now proven to be a physically realizable, correctly-timed circuit on the actual target chip (Xilinx Zynq-7020), not merely a design that looked reasonable on paper or in simulation.

The only remaining step this project has not attempted is physically loading this bitstream onto a real ZedBoard and confirming it correctly classifies live camera or pre-loaded image data on actual hardware —

which requires physical board access, separate from anything achievable through software tools alone.

Part 9: Rebuilding for the Actual Target Board (PYNQ-Z2)

After the bitstream described in Part 8 was generated, the project's guide asked a specific, important question before approving moving forward: which exact board and chip was this built for? This was a genuinely necessary check, and it uncovered a real mismatch that needed correcting before the bitstream could be considered final.

Why a "same chip family" bitstream can still be wrong for the actual board

The chip inside a PYNQ-Z2 board and the chip this project had been targeting are both a "Xilinx Zynq-7020" — the same underlying computational fabric, same amount of LUTs, flip-flops, DSPs, and block RAM. However, physical chips are manufactured in different **packages** — the actual physical housing and pin layout — and a Zynq-7020 in a 484-pin package is genuinely a different physical part from a Zynq-7020 in a 400-pin package, even though the logic inside is equivalent. **A bitstream is compiled for one specific physical package and cannot be loaded onto a different one.** This project's earlier bitstream (Part 8) had been built for the 484-pin package (matching boards like the ZedBoard); the PYNQ-Z2 board actually uses the 400-pin package. Despite being logically the "same chip," the earlier bitstream would not have been loadable onto a real PYNQ-Z2 board at all.

Getting the correct board information into the tools

Vivado does not automatically know the specific wiring and configuration details of every possible board a chip might be mounted on — for that, it relies on a small file (a "board file") published by the board's

manufacturer, describing exactly how the memory, clocks, and other components on that specific board are wired to the chip. The PYNQ-Z2's manufacturer (TUL Corporation, working with the open-source PYNQ project) publishes this file separately from Xilinx's own default installation. This file had to be located from its genuine, correct source and installed into Vivado before the tool could correctly configure the design for this specific board (in particular, the correct timing settings for the board's actual memory chip, which are load-bearing details that would be incorrect if left at generic defaults).

Confirming the existing design didn't need to be redesigned

Before rebuilding anything, it was confirmed that the seven-layer network's own hardware description (the `lenet5_top` design from Parts 4 and 7) is written generically enough that it does not depend on the specific package or board at all — only on the general "Zynq" chip family. This meant the actual computational design did not need to be changed or re-verified in any way; only the final, board-specific implementation and bitstream generation steps needed to be redone, targeting the correct physical part.

A real, unrelated problem discovered and fixed along the way

While this rebuild was in progress, a completely unrelated file belonging to this project (the very first convolution layer's source code, from Part 4) was found to have been accidentally overwritten with unrelated text from a different, unconnected piece of work happening elsewhere on the same computer at the same time — evidently the result of two separate work sessions unintentionally writing to the same file location. This was caught immediately via a routine check of what files had recently changed, the corrupted content was not acted upon in any way, and the correct, original file was recovered from the project's version-control history, where every previous correct version remains safely stored. This is worth recording honestly as a real lesson about running multiple work sessions on the same shared project folder at the same time: it's

possible for completely unrelated work to accidentally collide, and routinely checking what has actually changed before trusting or building on top of it is a valuable habit, not unnecessary caution.

The corrected result, for the real board

With the correct board file in place and a new, separate build specifically targeting the PYNQ-Z2's actual chip (`xc7z020c1g400-1`), synthesis, implementation, and bitstream generation were re-run in full, and completed successfully with zero design-rule-check errors.

Metric	Original build (484-pin package)	PYNQ-Z2 build (400-pin package)
Worst setup slack (WNS)	+3.659 ns	+0.033 ns
Worst hold slack (WHS)	+0.008 ns	+0.020 ns
LUT usage	55.3%	~55% (essentially identical)
FF usage	38.6%	~39% (essentially identical)
DSP usage	10.9%	~11% (essentially identical)
BRAM usage	85.4%	~85% (essentially identical)

Both builds pass timing, but it's worth being fully honest that the PYNQ-Z2 build's margins are noticeably tighter, especially the setup slack (+0.033ns, compared to the other build's +3.659ns) — both are genuinely positive and passing, but the PYNQ-Z2 result leaves very little room for error. This is a real, measured characteristic of this specific board's implementation, not a flaw in the design itself (the underlying logic and resource usage are essentially unchanged between the two builds) — most likely explained by the different package's slightly different physical layout leading to a different, somewhat less favorable

arrangement of the same circuit during automatic placement. If this design is ever revisited, it would be worth re-running implementation once or twice more to see whether a different automatic placement attempt yields more comfortable margins, since Vivado's placement process is not perfectly deterministic between runs.

Status: a genuine, board-correct bitstream now exists

This project now has a bitstream specifically and correctly built for the actual target board named by the project's guide (PYNQ-Z2), with confirmed, passing timing and resource utilization that comfortably fits the chip's real capacity. This is the version that should actually be used if and when physical hardware becomes available, rather than the earlier 484-pin-package bitstream from Part 8, which remains a valid demonstration of the same design but was never actually loadable onto this specific board.

Part 10: Converting From Floating-Point to Real Integer Arithmetic (INT8/INT32)

Everything described up to this point — every layer, the full combined network, both bitstreams — uses standard 32-bit floating-point numbers (`float`) throughout. This is the same kind of number Python and most everyday calculators use: capable of representing a huge range of values with fine precision, but comparatively expensive for an FPGA to do arithmetic with, since floating-point circuits are inherently more complex than plain integer circuits.

Following further guidance from the project's guide, the next requirement was to convert the design to use **real integer arithmetic in the actual hardware** — specifically, 8-bit integers for the weights and activations flowing between layers, and a wider 32-bit integer as the "running total" inside each layer's multiply-accumulate calculations (a wider accumulator is needed here for the same reason a calculator needs more digits of display than any single number you type into it — adding

up many 8-bit products can produce a result too large to fit back into 8 bits until it's deliberately rescaled down again).

This is an important distinction from earlier work in this project: the project had already explored 8-bit integer quantization extensively, but only as a **software simulation**, in Python — measuring what *would* happen to the network's accuracy if this representation were used, without ever actually building real integer-only circuits. This new phase asks for that same idea to be carried into the *actual synthesized hardware* for the first time.

Building and verifying the first converted layer

Rather than converting all seven layers to integer arithmetic simultaneously, only the first convolution layer (C1) was converted first, deliberately kept as a new, separate file rather than modifying the working floating-point version — preserving the same "never overwrite a proven design, always add a new one to compare against" discipline used throughout this project. This meant that if anything went wrong with this conversion, the fully working, already-verified floating-point network (including both generated bitstreams) would remain completely unaffected and safe.

Correctness was verified to an unusually strict standard. Because this is now genuine integer arithmetic rather than floating-point calculation, there is no room for the small, expected rounding differences that were considered normal and acceptable everywhere else in this document (recall Part 7's "difference of about 65 trillionths" being explicitly fine, since floating-point math legitimately has multiple valid orders of computation). Integer arithmetic has no such excuse — two implementations of the same integer calculation should produce **exactly, bit-for-bit identical results**, with zero tolerance for any difference at all. This layer's hardware output was compared against the equivalent Python calculation across every one of its 4,704 individual output values, and matched with **zero mismatches** — a stronger, more exact confirmation of correctness than anything else in this project, appropriate to the stricter nature of integer math.

The results: a genuine trade-off, not a simple win

Converting to 8-bit integers produced a real, substantial reduction in three of the four main hardware resource categories, exactly as the technique is generally expected to deliver — smaller data values simply need less physical circuitry to store and move around:

Resource	Floating-point version	Integer version	Change
On-chip memory (BRAM)	18 blocks (6%)	5 blocks (1%)	72% less
Memory bits (FF)	28,886 (27%)	9,989 (9%)	65% less
General logic (LUT)	19,572 (36%)	16,519 (31%)	16% less
Multiply-hardware (DSP)	14 (6%)	13 (5%)	Barely changed — flagged as worth investigating further

However, one genuinely unexpected and undesirable result also appeared: the design got slower, not faster — total calculation time increased by roughly 94%, and (separately) the floating-point version's timing had actually been failing to meet the clock-speed target, while the integer version succeeded in meeting it. This needed proper investigation rather than being accepted or dismissed without explanation.

Diagnosing why integer arithmetic became slower

The cause was tracked down to a specific, well-understood detail: even though the core multiply-accumulate calculation is now genuinely done in cheap integer arithmetic, the *very last step* of the calculation — converting the accumulated integer total back into a properly-scaled real-world value, so it can be correctly compared and passed to the next

layer — still relies on a **floating-point division** operation. A single hardware division is a comparatively slow, expensive operation to compute (this project has encountered exactly this kind of cost before, in Part 3's discussion of accumulator latency, though this is a related but distinct issue — division specifically, not simply accumulation) — and having it appear once per output pixel prevented the surrounding calculation from being pipelined as efficiently as the floating-point version had been, becoming the new limiting factor on overall speed.

The proposed fix, not yet completed at time of writing, is a standard, well-known hardware technique: instead of dividing by the same value repeatedly inside a loop, that value's *reciprocal* (one divided by it) can be calculated just **once**, ahead of time, and then every subsequent step can *multiply* by that reciprocal instead of dividing — multiplication is a much cheaper, more easily pipelined operation than division, even though the two approaches are mathematically equivalent. This fix has been identified and explained but deliberately not yet applied, since — following the same strict "bit-for-bit, not just approximately equal" verification standard established for this integer work — switching from division to reciprocal multiplication can, in principle, produce very slightly different floating-point results even though the two calculations represent the same mathematical operation, so this change needs the same rigorous re-verification against the Python reference before it can be trusted, rather than being assumed safe.

A second detail also flagged as worth further investigation: the multiply-hardware (DSP) usage barely changed between the floating-point and integer versions (14 down to just 13), which is a smaller improvement than might be expected — Zynq-family chips are generally able to pack small integer multiplications more efficiently into their DSP hardware than full floating-point multiplication requires, so a larger reduction was anticipated. This has been flagged for further investigation to determine whether some of the requantization step's remaining floating-point arithmetic (the scale-factor multiplication/division discussed above) is still consuming DSP resources that a fully integer-only pipeline should not need at all.

Status at time of writing

- Correctness of the first converted layer (C1): **confirmed**, to an exact, bit-for-bit standard, against a real trained model's actual weights, across three successive refinements described below.
- Resource savings (BRAM, FF, LUT): **confirmed and substantial**, as expected for this technique.
- Speed and remaining DSP usage: **fully resolved** — see the two follow-up refinements below.
- Remaining six layers: **not yet converted** — this conversion is intentionally being proven correct and well-understood on one layer first, following the same incremental approach used successfully throughout this project, before being extended to the rest of the network.

First refinement — replacing division with multiplication by its reciprocal

The first proposed fix from above was carried out: the repeated floating-point division inside the per-pixel calculation was replaced with a single, one-time calculation of that value's reciprocal, followed by ordinary multiplication in its place everywhere the division used to happen.

Critically, this change was not assumed to be numerically identical to the original — it was checked. Directly comparing the two approaches at the level of individual numbers showed that they genuinely do produce very slightly different results in a real portion of the data (roughly 11% of all values differed, by an extremely small amount, several millionths at most) — confirming this substitution is a genuine change in arithmetic, not a "free," perfectly invisible swap, even though the two operations are mathematically equivalent in ordinary, unlimited-precision arithmetic. Fortunately, this tiny divergence was confirmed to never actually change which final whole-number answer any value rounds to, so the overall calculation remained exactly correct — but this was established by direct checking, not assumed, and the Python reference calculation was correspondingly rebuilt to use the identical reciprocal-based arithmetic, so that the comparison being tested remained a fair, matching one.

This change alone reduced total calculation time by about 20%, while leaving resource usage essentially unchanged, and preserving the

design's ability to meet its clock speed target.

One further, precise finding came out of investigating why multiply-hardware (DSP) usage had barely improved earlier. A detailed look at exactly which operations were still using DSP hardware revealed that the core multiply-accumulate array had, in fact, already become fully integer-based and used a sensible number of DSP units for its size — but several *additional* DSP units were still being consumed by leftover floating-point operations specifically inside the final rescaling step (the scale-factor multiplication, and a separate step handling correct rounding). This precisely explained the earlier puzzle, and pointed directly at the next, final refinement.

Second refinement — genuine fixed-point rescaling, eliminating floating-point entirely

The final refinement replaced the floating-point rescaling step with a **true fixed-point** equivalent — a well-established, standard technique used in real commercial quantized-inference frameworks (the same general approach used inside Google's TensorFlow Lite, among others), where an arbitrary scaling factor is approximated not by a floating-point number at all, but by a specially chosen large whole number paired with a matching "shift" amount — such that multiplying by that whole number and then shifting the result produces the same effect as multiplying by the original scale factor, using only integer operations throughout, with no floating-point hardware involved anywhere in the calculation.

The correctness of this substitution was, once again, verified rather than assumed, following the exact same rigorous process established for every step of this conversion: a Python version of this exact fixed-point arithmetic was built and checked first — including an explicit, automatically-verified check that reconstructing the original raw calculation from the new representation produces an exact match — before any of this logic was ported into the hardware description at all. The hardware version was then confirmed to match this new Python reference with, again, zero mismatches across every single output value.

The result closed every remaining gap, and the new integer version

now outperforms the original floating-point design in every single measured respect:

Metric	Original (floating-point)	Integer, first attempt	Integer, final version
On-chip memory (BRAM)	18 (6%)	5 (1%)	5 (1%)
Multiply-hardware (DSP)	14 (6%)	13 (5%)	11 (5%)
Memory bits (FF)	28,886 (27%)	9,950 (9%)	9,889 (9%)
General logic (LUT)	19,572 (36%)	16,461 (30%)	16,124 (30%)
Calculation time (cycles)	144,591	223,780	124,991
Meets clock speed target?	No — failed by 2.36 ns	Yes	Yes
Maximum clock speed	104.80 MHz	136.99 MHz	136.99 MHz

The final integer version is not just smaller (as expected from switching to a narrower number format), but also genuinely **faster** than the original floating-point design (a 13.6% reduction in total calculation time) — and, notably, the *original floating-point version had actually been failing to meet its intended clock-speed target*, something not previously highlighted in this document, while every integer version successfully meets it. This is a strong, complete, and pleasantly surprising result: for this specific layer, switching to integer arithmetic turned out to improve every single property being measured, not merely trade size for speed or vice versa.

A final, precise explanation for the last few DSP units — a natural remaining question, given the above table still shows a non-zero DSP count even in the fully fixed-point version — is worth recording clearly: those remaining DSP units are not leftover floating-point circuitry at all (the synthesis tool's own report confirms literally zero floating-point hardware cores exist anywhere in this final design). They are needed

simply because the rescaling calculation still requires multiplying two reasonably large whole numbers together, and multiplying sufficiently large numbers together is an inherently DSP-costing operation on this chip regardless of whether those numbers represent an ordinary integer or a repurposed floating-point value — a subtle but important distinction: **DSP hardware cost is fundamentally about the width of a multiplication, not about which number format that multiplication happens to represent.**

Status, updated

- Correctness: **confirmed at every stage**, to an exact, zero-mismatch standard.
- Resource usage, speed, and timing closure: **fully resolved** — the final integer version outperforms the original floating-point design in every measured category.
- Remaining six layers: still **not yet converted** — the same three-stage refinement process demonstrated here (naive floating-point rescale → reciprocal multiply → genuine fixed-point) provides a proven, well-understood template ready to be applied to the rest of the network next.

A full, detailed numerical record of every individual experiment described in this document — including every attempt not elaborated on here in full narrative form — is maintained separately in the project's `Results/hls_results.md` file, which contains the complete, chronological, per-experiment data this summary is drawn from.

Extending the conversion to the rest of the network

With the full three-stage technique proven and well understood on the first layer, the same approach — build and verify a Python reference first, confirm it matches an independent calculation exactly, only then translate the identical logic into the hardware description, and verify that translation is itself exact before trusting any resulting measurement — was repeated for each of the remaining layers in turn: both pooling layers, the second convolution layer, and two of the three fully-connected layers.

Every single one of these conversions produced the same broad result already established for the first layer: substantial reductions in on-chip memory and register usage, and calculation speed that matched or improved upon the original floating-point version — with one further layer (the second convolution layer) also successfully carrying forward its own, separately-developed optimization (the earlier division-avoiding lookup-table technique from Part 4) *together with* this new integer conversion, confirming the two different kinds of optimization developed independently in this project are fully compatible with one another rather than working against each other.

A further, genuinely interesting finding: solving one bottleneck can reveal a different one

While converting one of the fully-connected layers, a finding emerged that is worth explaining carefully, since it extends this document's earlier central discovery (Part 3) into new territory rather than simply repeating it.

Recall from Part 3 that ordinary floating-point addition takes several clock cycles internally to produce its result, and that this specific cost was identified as the reason several layers' calculations could not be sped up past a certain point (an "Initiation Interval," or II, of 4), no matter how the calculation was restructured — this was described as a genuine hardware floor for floating-point arithmetic.

Plain integer addition does not have this same limitation — adding two whole numbers together is a substantially simpler, faster operation for hardware to perform than adding two floating-point numbers, and does not carry the same multi-cycle internal delay. Consequently, once this particular fully-connected layer's calculations were converted to integer arithmetic, the specific bottleneck that had been limiting its speed throughout this entire project — the floating-point addition delay — genuinely became a non-issue.

However, removing one bottleneck does not automatically mean a calculation reaches its absolute best possible speed — it simply means whatever the next-most-limiting factor happens to be will now determine the result instead. In this case, once the floating-point

addition delay was no longer the limiting factor, the *original* kind of limitation encountered all the way back at the very beginning of this project's optimization work resurfaced: a memory array with only a small, fixed number of ports available for reading multiple values in the same clock cycle (see Part 2's explanation of `ARRAY_PARTITION`, and Part 4's account of C1's very first optimization attempt). This had never mattered for this particular layer while the floating-point addition delay was the dominant cost — but with that cost removed, it became the new, visible limit.

This is a valuable, general lesson worth stating plainly: **fixing the most significant bottleneck in a design does not mean a design has no more bottlenecks — it means the *next* most significant one is now what determines performance, and that next bottleneck may belong to a completely different category of problem than the one just solved.** This project independently rediscovered this principle in two different forms — first when switching optimization *techniques* revealed a different limiting array (Part 4), and now again when switching *numeric representations* revealed a different limiting factor entirely. Both instances reinforce the same underlying truth about how hardware performance bottlenecks should be investigated: one at a time, by direct evidence, never assumed to be fully resolved just because a single fix improved things.

Status, updated again

- Correctness across every converted layer: **confirmed**, to the same exact, zero-mismatch standard established for the first layer, with each layer's own scale-conversion behavior independently derived and verified rather than reused from another layer.
- Resource usage, speed, and timing closure across every converted layer: **matches or improves upon** the corresponding floating-point version in every case measured so far.
- Compatibility with this project's other, independently-developed optimization techniques (specifically, the second convolution layer's division-avoiding lookup-table technique from Part 4): **confirmed compatible**, working correctly together in combination.
- Remaining work: the final fully-connected (classification) layer has

not yet been converted, and is expected to need additional care, since — as discussed in Part 4 — its final "softmax" calculation step has a fundamentally different shape from every other layer's calculation (it requires knowing every output value before correctly finishing any single one of them), meaning the straightforward integer-conversion technique used for every other layer cannot simply be copied without first thinking through how that specific final step should be restructured.

The final layer, and a precisely-explained exception to the "everything shrinks" pattern

The seventh and final layer (the network's classification output, ending in the "softmax" calculation first discussed in Part 4) was converted last, using the same careful, "think before implementing" approach as everywhere else in this project. A deliberate design decision was made and explained before any code was written: the raw multiply-accumulate portion of this layer was converted to genuine integer arithmetic, exactly like every other layer — but the final softmax normalization step was **deliberately kept in floating-point**, rather than being forced into the same integer/fixed-point treatment as everything else.

The reasoning is worth stating plainly, since it illustrates good engineering judgment rather than a shortcut: the fixed-point rescaling technique used throughout this section exists specifically to avoid the cost of floating-point arithmetic on a calculation that repeats very many times, once for every single output value, across every single input image — exactly the situation in a convolution or dense layer's main calculation. Softmax, by contrast, runs only **once per classification**, on only ten values total — it is the network's very last step, read directly by whatever system or person is interpreting the result, not fed forward into another layer that would benefit from receiving an already-quantized integer value. Given this, converting it to fixed-point arithmetic would have added real design complexity and risk for a calculation that was never a meaningful contributor to the design's overall size or speed in the first place.

This layer was the one place in this entire integer-conversion effort where a resource count increased rather than decreased —

specifically, the amount of multiply-hardware (DSP) used went up slightly. This was not accepted as a mysterious side-effect; it was traced precisely, using the tool's own detailed internal reports, to three distinct, separately-understood causes: a genuine small saving in the main calculation (fewer DSP needed there, as in every other layer), an accounting relocation with no new hardware at all (an existing calculation's cost simply got counted in a different part of the design than before, because a floating-point calculation elsewhere in the same layer that it used to share hardware with no longer exists once that portion became integer-based), and one small, genuinely new piece of hardware needed to bridge the new integer calculation's result back into the floating-point value the unchanged softmax step expects. This last piece is a small, deliberate, and justified cost of the design decision explained above — not evidence that combining integer and floating-point arithmetic in one layer somehow doubles the real work involved.

Part 11: Combining All Seven Converted Layers, and a Second Complete Bitstream

With every individual layer's integer-arithmetic version built and verified, the same combination step already demonstrated in Part 7 for the original floating-point design was repeated for these seven new integer versions — chaining all seven into a single, complete network capable of taking in one handwritten digit image and producing a full classification result, entirely in integer arithmetic (aside from the one deliberately floating-point final step discussed above).

Verifying the combined design was correct, at every internal stage

Rather than checking only the network's final answer, the combined design's testbench checked **every individual layer's output inside the chain**, not merely the end result — confirming zero mismatches at every stage from the first layer through the sixth, and confirming the final, deliberately floating-point classification stage matched to the same

small, well-justified tolerance used throughout this integer-conversion work. This is a stronger verification standard than checking only the final answer, since it confirms every internal handoff between layers — including each layer's own, independently-derived scale-correction values — is being carried through correctly, not merely that any errors happened to cancel out by the time the final answer was reached.

Fitting on the chip — this time, comfortably, without complication

Unlike the original floating-point combined design (Part 8), which required a real, documented detour (removing an attempted parallel-processing feature, then further optimizing one layer's internal arithmetic) before it could be made to fit within the target chip's capacity, this integer-based combined design **fit comfortably from the very first attempt**, using less than three-quarters of the chip's general logic capacity and well under a quarter of its on-chip memory — with no detours or additional optimization work required at all. This is a direct, practical demonstration of exactly the benefit integer arithmetic was expected to provide at the level of an entire network, not just one layer in isolation.

One further honest finding worth recording precisely

Despite every individual layer's own computation logic being substantially smaller in its integer form, the *combined* design's general logic (LUT) usage barely changed compared to the original floating-point combined design — a result that, on first glance, seems to contradict everything else found in this section. This was investigated properly rather than left unexplained.

The cause turned out to be specific and well understood, not a flaw in the conversion technique itself. Each layer's connection to the rest of the system (the same kind of memory connection discussed in Part 2) had, for simplicity, been set up to carry both a layer's weight values and its bias values together over one shared connection — a perfectly reasonable choice when those values were both the same, 32-bit floating-point format, as in the original design. Once the weight values

became a much narrower 8-bit integer format while the bias values remained a wider format on the very same shared connection, the hardware's ability to efficiently pack multiple narrow values together when moving them to and from memory was lost, adding real, if avoidable, extra circuitry at exactly the boundary between each layer and the rest of the system. Summing just the computational core of all seven layers directly from the same report confirms their combined logic usage is, in fact, dramatically smaller, exactly as expected — the increase is entirely attributable to this specific, identified connection-sharing inefficiency, not to the integer arithmetic itself being larger than expected.

This has been recorded as a known, well-understood, and low-priority opportunity for further improvement (separating each layer's weight and bias values onto their own, correctly width-matched connections would be expected to recover a meaningful portion of this cost) — but, since the combined design already comfortably fits the chip's capacity without this fix, it was deliberately not pursued immediately, in favor of completing and verifying the full, working hardware result first.

The final, complete result: a second real bitstream

Following the exact same real-hardware validation process used for the original floating-point design (Part 9) — sourcing and confirming the correct PYNQ-Z2 board file, configuring the processor's high-performance memory connection, and working around the same known tool limitation with this design's many independently-named memory connections by wiring them explicitly rather than relying on automatic connection — a complete, real, physically valid bitstream was generated for this integer-arithmetic version of the network as well.

One additional, genuine practical obstacle was encountered and resolved along the way: partway through this process, file generation began failing due to a length limit built into the Windows operating system itself, restricting how long a complete file path is allowed to be. This project's file and folder naming had, by this stage, grown descriptive enough (reasonably so, for clarity) that combined with this new design's own internal file-naming conventions, a small number of generated files ended up with names exceeding this limit. This was confirmed precisely,

by directly measuring the problematic path length against the equivalent, working path from the original floating-point build, rather than guessed at — and resolved simply by shortening the new project's own folder and file names, which is a purely administrative fix with no bearing whatsoever on the correctness of the underlying design.

Final, real, place-and-routed results, compared directly against the original floating-point bitstream:

Metric	Floating-point bitstream	Integer-arithmetic bitstream	Change
Worst setup slack (WNS)	+0.033 ns	+0.121 ns	More comfortable margin
Worst hold slack (WHS)	+0.020 ns	+0.015 ns	Still positive, similarly tight
General logic (LUT)	55.3%	31.8%	42.5% less
Memory bits (FF)	38.6%	21.2%	45.1% less
On-chip memory (BRAM)	85.4%	25.7%	69.9% less
Multiply-hardware (DSP)	10.9%	20.9%	Higher — expected, and precisely explained above and throughout this section

Every single timing requirement is met, with **zero failing paths**, for this integer version as well — and, notably, its timing margin is measurably more comfortable than the original floating-point design's had been, addressing the earlier, honestly-recorded concern (Part 8) about that design's very thin safety margin.

Status: two complete, working, verified bitstreams now exist

This project now has two full, real, physically-valid FPGA configurations for the complete seven-layer LeNet-5 network, targeting the actual PYNQ-Z2 board specified by the project's guide: one using standard floating-point arithmetic throughout, and one using genuine 8-bit integer arithmetic (matching the guide's specific request), verified correct against real trained weights and a real handwritten digit image, meeting every timing requirement, and — in the integer version's case — using substantially less of the chip's available hardware resources than the floating-point version required, while also running with a more comfortable timing margin.

A full, detailed numerical record of this and every other individual experiment described in this document is maintained separately in the project's `Results/hls_results.md` file.

Extending the conversion to the rest of the network

With the fixed-point technique proven correct and beneficial on C1, the same approach — build a Python reference first with its own independently-derived scale multiplier, verify that reference matches an independent cross-check exactly, only then port the identical arithmetic into HLS, and confirm an exact, zero-mismatch match before trusting any synthesis result — was repeated for each remaining layer in turn, chaining each layer's real output forward as the next layer's genuine test input (rather than artificial test values), so the whole growing pipeline was tested against realistic, real-world data at every stage.

Every layer converted so far has repeated the same encouraging pattern: substantial reductions in on-chip memory and general logic usage, equal or improved calculation speed, and — consistently, across every single layer — a design that had been *failing* to meet its clock-speed target in floating-point now *passing* that same target once converted to integer arithmetic.

One genuinely new and instructive finding appeared while converting the first fully-connected (dense) layer, C5. Recall from

Part 3 of this document that the project's central floating-point finding was that a slow, multi-cycle floating-point addition operation was the limiting factor capping how quickly these calculations could be pipelined. Integer addition, by contrast, is a much simpler and faster operation for hardware to perform, with no equivalent multi-cycle delay. Once this layer's arithmetic was converted to integers, that specific limiting factor genuinely became less severe — and, as a direct result, a **different**, previously-hidden bottleneck was exposed instead: the same "not enough simultaneous memory read ports" limitation that this project first encountered and solved, right at the very beginning of this whole effort, in Part 4's discussion of the very first convolution layer.

This is a valuable, concrete illustration of a general principle in hardware design worth stating plainly: **solving one bottleneck does not guarantee reaching the best possible result — it only reveals whatever the next most limiting factor happens to be**, which may turn out to be a completely different *category* of problem than the one just solved. This project has now demonstrated this same principle twice, in two different ways: once by discovering that partitioning the wrong array produces no benefit at all (Part 4), and now by discovering that removing a slow-arithmetic bottleneck can, by itself, unmask an entirely different, previously-irrelevant memory-access bottleneck underneath it.

Summary of layers converted to integer arithmetic so far:

Layer	Resource change vs. floating-point	Speed change	Timing result
C1 (convolution)	Substantially smaller	13.6% faster	Was failing target; now passes
C3 (convolution)	Substantially smaller	6.0% faster	Was failing target; now passes
S2 (pooling)	Substantially smaller (comparison logic only)	Essentially unchanged	Already passing; still passes

S4 (pooling)	Substantially smaller (comparison logic only)	Essentially unchanged	Already passing; still passes
C5 (fully-connected)	Substantially smaller	46.3% faster	Was failing target; now passes
F6 (fully-connected)	Substantially smaller	36.0% faster	Was failing target; now passes
Output (final/softmax)	Mostly smaller, one small increase (explained below)	32.1% faster	Was failing target; now passes

All seven layers have now been converted and individually verified — the final layer (Output) is discussed in its own dedicated section immediately below, since, as expected going in, its "softmax" step needed a distinct design decision rather than the same simple "multiply-accumulate, then rescale" pattern used for the other six.